FORMAL VERIFICATION / REVIEWABLE REFACTORING

Sharing the K215 Proofs

A two-detachment basis and certified residual execution

Prepared for Vladimir Reshetnikov and the ProveIt project
21 October 2026 (Pacific time)

Abstract

The 215-state machine already has substantial checked formalizations. The next useful simplification is to share proofs at their mathematical interfaces, while retaining the exact machine, source program, and public correctness statements. This report develops two concrete changes against ProveIt commit `57541fe0cfe3af62b4cb58747d8d1d7c16b338d5`. First, three lemmas already present in the dominated-implication proof derive the implicational S schema with only two additional modus-ponens applications. Together with existing K and Peirce lemmas, this permits one abstract proof to serve the dominated numerical calculus and the unrestricted calculus. The selected Lean logic sources shrink from 42,454 to 18,096 bytes; the corresponding Rocq replacement also removes duplicate basis proofs. Second, the already checked decoder-prefix theorem is connected to a residual executor through a semantic site contract. The remaining finite checks retain every literal leaf and both decrement outcomes. A third, longer-term recommendation is to implement the previously proposed direct full-seed construction, eliminating an intermediate semantic quotient in Lean. We provide source changes, reproducible checks, exact measurements, and integration gates. Component verification is distinguished throughout from a new whole-endpoint rebuild, and no new internal arithmetization or machine-state reduction is claimed.

1 Recommendation and scope

Adopt the shared implication basis first, then the residual site checker. The first change removes a complete duplicate mathematical development while preserving familiar theorem names. The second changes where repeated execution is proved: a common decoder prefix is established once, and each finite site certificate checks the suffix that begins at the exact certified entry port. Both are incremental refactorings of the current proof. Neither requires changing the machine's calculus, its transition table, or its command enumeration.

The key new local mathematical observation in this packet is the two-detachment derivation of S in Section 4. An earlier packet already proposed sharing the implication core, with a five-detachment extension for S. We reuse and attribute that architecture rather than treating it as a new discovery. The additional contribution here is a smaller extension, checked against the current core, and a paired integration through both Rocq calculus interfaces. The residual work completes an earlier proposed interface by connecting the current source-binding theorem to executable suffix checks.

Change	Concrete benefit	Evidence boundary
Shared Lean basis	24,358 fewer proof-source bytes; 400 fewer lines	Fresh focused baseline and candidate closures, with typed public clients
Shared Rocq basis	One abstract proof used by both calculus adapters	Focused compilation, closed assumption reports and recursive component check
Residual site checker	The common decoder prefix leaves the per-site executor	Prefix/port soundness, semantic simulation and finite site checks
Whole K215 endpoint	Exact existing target retained as the integration target	Full production endpoint replays remain required for adoption

The archive contains all proposed source changes and the evidence needed to review their scope. Reported source savings concern named file sets. They do not measure the size of fully expanded proof terms or establish that a shortest possible derivation has been found. Timings record executions in this environment; historical repository timings are not used as a controlled speed comparison.

2 What is already present at the reviewed revision

2.1 Read source files before interpreting commit titles

The review is pinned to the exact Git commit above, acquired from the repository's default branch. This matters because the repository has many parallel proposals, merged reports, and checked checkpoints. A merge that retains a proposed file does not necessarily install it as a production proof dependency.

The phase-twelve checkpoint already separates the `ZFModels` and `WFModelCore` semantic interfaces from older clients. These are distinct from the proposed `KSim` semantic-coverage interface used below. Its selected Lean endpoint closure contains 179 project modules and 2,129,439 source bytes. The associated check covers 184 modules: the endpoint closure and five decoder-prefix modules. Fifteen providers were freshly compiled, while 169 were reused with exact source, import and object bindings. Two endpoint clients were freshly checked. The separate compatibility check is another scope, not evidence that all of these modules were rebuilt together [2].

The maintained `DominatedImplication.lean` contains the short generic dominated-row proof: 287 lines and 13,723 bytes. However, the maintained `Lukasiewicz.lean` still contains the independent 107-certificate development, at 524 lines and 28,731 bytes. The shared-basis alternatives in the proposal tree are therefore still actionable. Their presence and their commit descriptions should not be counted as an already completed reduction of the current source closure [3, 4, 5].

The Rocq evidence has a similar distinction. The original complete proof built 237 modules and recursively checked the dense endpoint. The later compact `DomRho8.v` checkpoint checks seven affected modules, reuses 230 exactly bound providers, and preserves closed endpoint assumption reports. Its focused recursive check is of the implication component. Those records should not be described as a fresh 237-module recursive endpoint replay of the later bytes. The independent snapshot audit in this archive records the precise matching source hashes [6].

2.2 The target that must survive refactoring

The dense K215 table has 215 working states, 430 binary transition rows, and a separate halt state. Its SHA-256 digest is

```
fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090
```

The transition convention and the table itself remain unchanged [1].

The Lean endpoint is

```
KEndpoint215.blank_halts_iff_zfc_refutation :
  BlankHalts machine215 <-> BProv ZFCax_s [] Form.fBot
```

where `<->` in this display denotes Lean's equivalence notation. Its nonhalting companion negates both sides. Rocq's dense endpoint is

```
K215F.CorrectDense215.tm215_halts_iff_ZFC_inconsistent :
  Dense.halts dense_tm Dense.c0 <-> StandardZFC |-TC falsity.
```

The display abbreviates the existing Rocq proof notation. These endpoint types are inherited integration targets. The newly executed typed audits in this packet check the changed component interfaces; they do not recheck the complete endpoint declarations.

These are external formal theorems relating the concrete computation to a finite formal refutation of ZFC. The project's ordinary mathematical calibration is intended to establish the corresponding assertion in ZFC. This packet preserves the external endpoints and their assumptions. It does not add the separate internal PA endpoint or a mechanically checked arithmetized ZFC proof of the equivalence. Those are different formalization tasks. In particular, simplifying the proof of the equivalence does not establish unconditional nonhalting.

3 One abstract implication proof

Let A be any type, let $a \Rightarrow b$ denote an arbitrary binary operation on A, and let $\mathsf{T}(a)$ mean that a is derivable. We assume modus ponens:

$$\mathsf{T}(a \Rightarrow b) \ \wedge \ \mathsf{T}(a) \quad \Longrightarrow \quad \mathsf{T}(b).$$

The operation need not be injective, and the elements need not decode to well-formed formulas. These are algebraic derivation statements, independent of a particular coding of syntax.

The maintained generic core additionally has a relation $\preceq$ satisfying

$$u \preceq u, \qquad u \preceq x \Longrightarrow u \preceq (x \Rightarrow y), \qquad u \preceq y \Longrightarrow u \preceq (x \Rightarrow y),$$

and grants the restricted Lukasiewicz row

$$u \preceq a \quad \Longrightarrow \quad \mathsf{T}(((c \Rightarrow u) \Rightarrow a) \Rightarrow ((a \Rightarrow c) \Rightarrow (b \Rightarrow c))). \tag{1}$$

No transitivity or antisymmetry assumption is imposed. The existing derivation establishes the unrestricted row, implication composition, and right lifting from this interface. We preserve the Lean core's entire private proof prefix byte for byte.

There are two useful interpretations of the relation. For numerical machine codes, use the already checked numerical domination relation and its growth lemmas. For an unrestricted calculus, use the universally true relation:

$$u \preceq a \quad :\Longleftrightarrow \quad \mathsf{True}.$$

Every guard is then trivial, and the unrestricted row supplies the restricted interface. This is an instantiation of a proved theorem, not a conversion of semantic truth into syntactic provability.

The earlier derivation already contains

$$\begin{aligned} &\texttt{d12}: \quad \mathsf{T}(a \Rightarrow (b \Rightarrow a)),\\ &\texttt{d18}: \quad \mathsf{T}(((a \Rightarrow b) \Rightarrow a) \Rightarrow a). \end{aligned}$$

These become public K and Peirce exports by direct aliases. They require no additional applications of modus ponens. A short derivation of S completes the basis required by the named-formula completeness development.

4 The two-detachment derivation of S

The following statement isolates the new contraction. Its proof does not depend on a theorem prover, a particular encoding, or the history of the larger dominated-implication derivation.

Theorem 4.1 (Two-detachment basis extension)**.** *Suppose* T *is closed under modus ponens and the following three schemas are derivable for all their arguments:*

$$\begin{aligned} \mathrm{D}_{34}(x,y,z,w) &: ((x \Rightarrow y) \Rightarrow (z \Rightarrow w)) \Rightarrow ((x \Rightarrow w) \Rightarrow (z \Rightarrow w)),\\ \mathrm{D}_{55}(x,y,z) &: (x \Rightarrow y) \Rightarrow ((z \Rightarrow x) \Rightarrow (z \Rightarrow y)),\\ \mathrm{D}_{57}(x,y,z,w) &: (x \Rightarrow y) \Rightarrow ((z \Rightarrow x) \Rightarrow (w \Rightarrow (z \Rightarrow y))). \end{aligned}$$

Then the schema

$$S(a,b,c) = (a \Rightarrow (b \Rightarrow c)) \Rightarrow ((a \Rightarrow b) \Rightarrow (a \Rightarrow c))$$

is derivable using two additional applications of modus ponens.

Proof. Write

$$X = b \Rightarrow c, \qquad U = a \Rightarrow X, \qquad V = (a \Rightarrow b) \Rightarrow (a \Rightarrow c).$$

The instance $\mathrm{D}_{57}(X, c, a, a \Rightarrow b)$ is

$$\mathsf{T}((X \Rightarrow c) \Rightarrow (U \Rightarrow V)).$$

The instance $\mathrm{D}_{34}(X, c, U, V)$ is

$$\mathsf{T}(((X \Rightarrow c) \Rightarrow (U \Rightarrow V)) \Rightarrow ((X \Rightarrow V) \Rightarrow (U \Rightarrow V))).$$

The first application of modus ponens therefore gives

$$\mathsf{T}((X \Rightarrow V) \Rightarrow (U \Rightarrow V)).$$

Finally, $\mathrm{D}_{55}(b, c, a)$ is precisely $\mathsf{T}(X \Rightarrow V)$. The second application gives $\mathsf{T}(U \Rightarrow V)$, the desired S instance. □

The exact Lean body is short enough to review directly:

```
h.mp
  (h.mp (d34 h (wim b c) c (wim a (wim b c))
    (wim (wim a b) (wim a c)))
    (d57 h (wim b c) c a (wim a b)))
  (d55 h b c a)
```

Here `h.mp` takes the major premise first. The code uses exactly the three existing theorem names in Theorem 4.1. The paired Rocq term applies the same two inferences. A separate Coq audit also proves the abstract statement with the three schemas and modus ponens quantified as explicit premises.

The identity was located by an untrusted first-order unification search over already present unconditional schemas. The search output is not trusted by either proof assistant: the resulting explicit term is checked again. We make no minimality claim. In particular, an unsuccessful one-step search in a selected schema pool would not rule out a different one-step derivation from a larger pool or a different factoring of the core.

4.1 What the proof-node count means

The old independent Lean named-calculus development contains 107 certificates: one axiom schema followed by 106 detachment steps. The maintained dominated core contains 58 shared detachment steps for its main goals. Counting these together gives 164. Adding the two-step S extension to the shared core gives 60. Thus the selected shared derivation count changes from 164 to 60, a reduction of 104.

The unchanged one-step `lift_right` rule is excluded from both figures. Including it gives 165 to 61. The earlier shared-core proposal used five extra steps for S, giving 63 rather than 60 under the same convention. These counts refer to shared schema definitions, not to the size obtained by inlining every proof into every caller.

The 164-to-60 baseline comparison is specifically a count of the Lean schema developments. The old Rocq dominated proof is a reflective replay with its own instruction count. Its replacement has the same 60-step abstract core, but the old replay instructions should not be conflated with the Lean schema count.

5 Lean implementation and preservation checks

The implementation makes three proof-source changes. First, it appends `basisK`, `basisPeirce`, and `basisS` to the maintained core. Second, it adds a pure `ImplicationalBasis.lean` adapter. Third, it replaces the duplicated certificate prefix of `Lukasiewicz.lean` with an import of that adapter. The Tarski bridge tail is byte-for-byte identical to the baseline.

The pure adapter retains the original namespace and the original `Implicational` structure, including constructor order and projections. Its language instance uses the universal relation, and its four exports retain the original public types of `k`, `peirce`, `composition`, and `s`. Downstream callers therefore continue to state the same theorems. The numerical `DomRho8.lean` adapter is unchanged.

Proof source	Old bytes	New bytes	Old lines	New lines
`DominatedImplication`	13,723	14,356	287	306
`Lukasiewicz`	28,731	1,667	524	45
`ImplicationalBasis`	0	2,073	0	60
Total	42,454	18,096	811	411

The 57.38% byte reduction includes the new adapter. Adding the unchanged numerical adapter to both sides changes the percentage but leaves the same 24,358-byte saving. The three-file change adds one module to the endpoint closure: 179 becomes 180 under this change alone. That is a smaller proof development with a slightly larger module count. It would be misleading to describe it as another reduction in the number of dependencies.

The maintained generator is synchronized rather than left to overwrite the new proof at the next regeneration. It still validates the exact original 58-step input and preserves the existing single-use inlining. It reads a pinned small basis extension, and its constructor-syntax checker validates all six public targets and 61 total modus-ponens applications when the unchanged lifting rule is included. Generated core and numerical adapter bytes exactly match the proposed sources. This Python diagnostic is useful for regeneration; the Lean kernel remains the authority for the proof.

Fresh baseline and candidate builds include the generic core, the named completeness bridge, the numerical calculus clients, and independently typed public-interface audits. The candidate has 15 project modules and two audit clients; the baseline has 14 project modules and two clients. The generic core and the unrestricted basis have no global axioms. The downstream semantic and numerical clients preserve their baseline named axiom profiles. Details are in the attached compiler logs and the comparison record.

6 Rocq implementation: share both calculus adapters

Replacing only the large `DomRho8.v` replay would leave the separate `CD222.v` basis development in place. The proposed Rocq change therefore routes both through the same abstract core. The restricted numerical adapter instantiates the growth relation with natural-number domination. The unrestricted `CD222` adapter instantiates it with the always-true relation.

The original `DomImp` interface is retained. The numerical unrestricted row theorem and the four `CD222` basis exports keep their original types. The current `ThmDom` client is compiled unchanged, so the comparison is not limited to an isolated polymorphic theorem with no connection to the actual calculus.

The numbered `cd_0` through `cd_106` implementation lemmas and the internal certificate replay API are retired. A census found no use of the numbered lemmas outside `CD222.v` in the maintained K215 sources. Consumers outside that checked scope that import those implementation names would need compatibility wrappers; the preserved interface is the consumed basis and `DomImp` API, not every historical helper name.

Selected Rocq proof sources	Old	Proposed
Bytes, including the new shared core	45,538	17,647
Lines, including the new shared core	1,399	371

The packet includes the corresponding regeneration and compile-order changes. The reviewed generator produces the new common module before its two consumers. A complete regeneration of the K215 lane emits 43 modules: 40 are byte-identical to the baseline, two are the intended replacements, and one is the new core. This is a necessary part of simplifying generated proof developments: a patch that changes only their current output is temporary.

Local validation uses Coq 8.18.0, the available checker, with explicit `Stdlib/Coq` import normalization where required, and CRLF-to-LF line-ending normalization. Both the baseline and candidate focused closures are compiled, their identical public-type audit is checked, and recursive `coqchk` is run. The resulting declarations are closed under the global context. The original pinned source text and normalization boundary remain visible in the package.

This is not a replacement for the repository's Rocq 9.1.0 endpoint gate. Production adoption must run the pinned toolchain, regenerate the maintained source lane, and rebuild the affected endpoint closure with its existing closed-assumption and recursive-kernel checks. A successful smaller component check establishes those components, not every imported endpoint artifact under a different toolchain.

7 Factor the decoder prefix from the site certificates

The maintained simulation checks 4,751 primitive leaf occurrences, including both guarded outcomes at each decrement. This amounts to 6,176 branch obligations. The original site checker starts at the dispatch configuration for every obligation. Consequently it symbolically executes the common decoder prefix repeatedly before it reaches the primitive action.

The existing `KDecoderPrefix215.walk_prefix` theorem is stronger than a table of guessed entry states. It takes an actual successful source walk, retains its complete consumed word including zero padding, and proves a uniform physical run to a symbolic leaf port for every gap and register parameter. The port contains the reverse of the full consumed prefix on the left of the head. It stops before the terminal primitive bit is processed. The source-binding and role-graph proof supplies its connection to the exact machine [7].

What was missing is a checked consumer of that interface. The new `KResidual215.lean` supplies it, through the following decomposition:

$$\text{dispatch at } Q \quad \longrightarrow^{*} \quad \text{exact leaf port} \quad \longrightarrow^{+} \quad \text{successor dispatch.}$$

The first relation is supplied by the reusable decoder theorem. The second comes from the residual symbolic executor. Their composition is the same positive physical realization needed by the existing simulation induction.

7.1 The semantic contract

A site contract records the conclusion that the simulator actually consumes. For a nonhalting branch it has the form

$$\forall n, vs, \quad \mathrm{Valid}(br, vs) \Longrightarrow \mathrm{conc}(n, vs, \mathrm{start}(Q)) \longrightarrow^{+} \mathrm{conc}(n, vs, \mathrm{target}). \tag{2}$$

It also retains the existing path-length and register-index bounds. The decrement contract includes both the zero and nonzero cases, under their separate branch guards. A halt contract retains the premise that the machine's terminal label has no outgoing transition.

The earlier semantic-coverage proposal already separated this contract from the particular Boolean checker that produced it. We retain that interface and its compatibility wrappers. The original checker remains one sound producer; the residual checker becomes a second. The simulation relation, its register encoding, and both directions of the halting equivalence are reused.

Proposition 7.1 (Prefix/suffix composition)**.** *Suppose an exact symbolic input I satisfies, for every n, vs,*

$$\mathrm{conc}(n, vs, \mathrm{start}(Q)) \longrightarrow^{*} \mathrm{conc}(n, vs, I).$$

Suppose a residual executor accepts the target from I, and its soundness theorem supplies positive reachability for every valid vs. Then the site satisfies (2)*.*

Proof. Fix n, vs satisfying the guard. Compose the finite run from the dispatch configuration to I with the positive finite run from I to the target. The sum of a nonnegative length and a positive length is positive. The halting case composes the same prefix with a finite run to a terminal configuration. The proof is uniform in the gap and register values. □

7.2 Why exact ports and progress matter

The implementation computes a port by following the certified role graph along the complete source prefix. It proves that this computation returns the state in the theorem's `RoutedEntry`. The physical state alone is insufficient: different roles can share a state, while the consumed word and the remaining program-counter material distinguish their continuations.

The checker therefore retains the full original word when constructing the successor path, overflow condition, and gap. It rejects an empty instruction word or a missing role-graph path. It does not substitute a short suffix for the consumed program counter. Those requirements prevent a compact but incorrect proof that accidentally changes the tape representation.

Every residual executor starts with its progress flag set to `false`. Acceptance then supplies a positive suffix run. This is a deliberate invariant, not a performance detail. If arbitrary zero-step matching were accepted as the realization of a source step, an endlessly repeating source configuration could be related to an already halted concrete configuration. The forward simulation would make no physical progress, and the reverse halting argument would fail. The inherited `Reach1` interface prevents that collapse.

7.3 Finite coverage and checked scope

The proposed site generation keeps the original nine batches and all 105 subtree boundaries. Their encoded source subtrees are unchanged; only the checker and its soundness bridge change. The final composition obtains semantic coverage from the all-sites result, then invokes the existing semantic simulation theorem. In particular, the role-graph computation is not used as an unproved replacement for source membership.

A fresh Lean 4.32.0 build compiled 34 project modules and one typed audit client, including all nine residual batches and their final composition. All checks passed. A separate kernel calculation over the literal source tree establishes exactly 4,751 leaves and 6,176 guarded outcomes. The old and new typed runtime simulation clients both report precisely `propext`, `Classical.choice`, and `Quot.sound`; the finite all-sites theorem reports `propext` and `Quot.sound`. The source, import-object, output-object and log bindings are recorded in `residual_sites/`. These checks establish the runtime component with its original engine and terminal-state premises; they do not recompile the complete initialization, source/calculus, and ZFC endpoint closure.

The selected runtime sources grow from 86,896 bytes and 1,321 lines to 103,551 bytes and 1,651 lines, before the audit client: an increase of 16,655 bytes and 330 lines. This comparison includes the simulator interface, the new residual bridge, and all site batches and their facade. It compares replacement footprints; the supplied patch uses new site-module names and retains the old route for compatibility. Its purpose is to reduce repeated proof computation and make the proof architecture reusable. The exact finite coverage remains; we do not claim that the 6,176 obligations have been reduced to a smaller number merely by giving their common prefix a name. Likewise, no overall endpoint speedup is inferred from a historical timing collected under different conditions.

The separate `endpoint-UNCHECKED.patch` shows how to connect the checked runtime component to `KEndpoint215`. Source review confirms that it retains the exact halting/refutation and nonhalting/consistency statement types displayed above. That endpoint adapter has not been compiled in the complete source, boot, and logic closure. It is an integration proposal, not a replacement endpoint receipt.

8 A deeper next step: a direct full-seed construction

After the checked changes above, the most promising mathematical simplification is to implement the repository's existing `DIRECT_FULL_SEED.md` proposal. It addresses a different source of complexity: the lower consistency argument first obtains a seed for equality-free formulas, then takes an indiscernibility quotient to recover literal equality. The proposed direct bridge would provide the full seed on the original domain. This is prior work whose compatibility with the current interfaces has been reviewed here; no new compiled version is included [8].

The decisive distinction is in `KEind222.c222_complete`. Its syntactic premise concerns codes `enc (th f)`, whereas the model assumption inside its semantic no-model premise satisfies the original named formula f, with literal equality. Its proof does not depend on the seed or indiscernibility modules, so using it for this construction introduces no circular appeal to completeness.

For any named formula ψ, define

$$p = \neg\neg\psi, \qquad c = \mathrm{enc}(\mathrm{th}(p)), \qquad B = \mathrm{bundleNF}(0, 1, c, p).$$

Two structural inductions give: every free name of f is below $\mathrm{enc}(f)$, and interpretation `th` preserves free names. Consequently c cannot be free in p: otherwise $c < \mathrm{enc}(\mathrm{th}(p)) = c$. The double-negation padding also gives $c > 1$, separating the private label from the two fixed roles. Interpretation commutes with the bundle syntax, so B belongs to the current `DerivableNF` theory. A general named-bundle semantics lemma then yields the seed property for p, and double-negation elimination gives it for ψ. This works even when ψ contains equality.

The desired replacement has the same premises as the present bridge, but a stronger and simpler conclusion:

```
fullSeed_direct_of_derivable mem hirr hT : FullSeed mem
-- Current bridge concludes: FullSeed (qmem mem)
```

The endpoint can then use the already available witness `e 0`, avoiding the intermediate quotient representative `mk mem (e 0)`. The downstream tagged domain, bisimulation quotient, well-founded construction, and ZF model argument remain in place. This change concerns only the first quotient.

An import-graph review identifies `KIndisc222`, `KSeedModel222`, and `Code254Models` as an exclusive three-module branch that could be removed from this endpoint path after the bridge and imports are reorganized. Their current combined size is 15,508 bytes and 339 lines. This is a gross candidate

removal, not a net saving: the support lemmas, named-bundle semantics, and direct bridge must first be implemented and counted. The small `FullSeed` interface must remain accessible. The existing numerical bundle theorem must also remain for its other consumers, including degenerate code and role cases.

The corresponding Rocq proof already follows an analogous direct route. Thus this proposal would bring the Lean and Rocq mathematical architectures closer together; it does not imply the same deletion in Rocq. Its acceptance gate is a compiled direct bridge, the unchanged final endpoint types and axiom profiles, and an actual dependency and size comparison after the new lemmas are included.

9 Integration plan and acceptance gates

Apply the logic and runtime changes as separate reviewable steps. They address different costs and need not be merged together to obtain a useful result.

1. **Review the two-detachment term.** Check the three schema instances in Theorem 4.1, the exact public type audits, and the absence of new global axioms. Retain the current generic proof prefix.
2. **Apply the Lean basis overlay and regenerate.** The supplied patch includes the source generator and its pinned extension. Require exact regenerated bytes and compile the unchanged numerical and semantic clients.
3. **Apply the Rocq basis overlay independently.** Review the new shared core, both adapters, generator templates, and compile order. Check the complete generated-file comparison before updating production receipts.
4. **Review semantic coverage before switching site producers.** Confirm that all original executable definitions and public wrappers retain their intended calling conventions. Review the exact port theorem and the positive-progress invariant separately from finite computation.
5. **Check every residual site.** Retain all nine batch results and the typed final simulation client. Both decrement branches must survive even when they share a decoder prefix.
6. **Run production endpoint gates.** Compile the affected transitive descendants of the exact K215 endpoint, use source-bound reuse only through the repository's existing mechanism, recompile both final typed clients, and compare their named axiom profiles. On Rocq, use the pinned 9.1.0 environment and its recursive endpoint check. Only that result should replace the maintained acceptance receipts.

The archive contains overlays and patches rather than silently editing the upstream branch. Patches are checked against the pinned baseline. A later repository revision may require a small reconciliation; in particular, do not replace a newer checked proof with an older copy simply to make a patch apply. The mathematical interface and the before/after hashes identify what needs to be carried forward.

10 Further research questions

1. **Can the implication basis be smaller still?** The two-step extension is sufficient, but there is no lower-bound proof. Search can optimize the union of the unrestricted row, K, Peirce, composition, and S goals, rather than optimizing one goal and adding the others afterward. Any improvement should be rendered as explicit kernel terms and compared with the exact same set of public targets.

2. **Can primitive suffixes be shared symbolically?** Prefix factoring still leaves a residual check at every leaf. A stronger result would give uniform contracts for increment, decrement, break, continue, and return families, parameterized by the full continuation. The difficult part is maintaining the tape frame and carry/overflow behavior across arbitrary consumed prefixes.
3. **Which representation minimizes kernel work?** Compare explicit terms, reflective certificates, and theorem applications using both source size and kernel execution cost. A smaller generated text can elaborate to a larger term; a new abstraction can add source while removing repeated computation. Measurements should isolate those effects.
4. **Can Rocq adopt the same semantic site interface?** The prefix/suffix composition argument is independent of Lean. A Rocq port should expose the actual positive-reachability contract and connect it to the current source/tree representation before attempting to reuse finite data from the Lean lane.
5. **How far should compatibility wrappers be retained?** Keeping old names makes adoption safer. Once all clients are migrated, some wrappers and imports may be retired. That is a dependency-management change and should be evaluated separately from the mathematical contraction.
6. **What remains for the internal consistency endpoint?** The external halting/refutation theorem is the invariant of these changes. Formalizing its arithmetization requires an explicit coding of runs, syntax, proof predicates, and translations. A short external proof is valuable preparation, but it does not automatically discharge that task.

11 Reproduction, attribution and limitations

The top-level `README.md` lists the portable component commands. The integrity checker verifies the archive's declared file digests. It is not a proof checker. The Lean driver creates fresh baseline and candidate workspaces, checks the pinned source revision, compiles their dependency closures with the pinned Lean version, and records source/import/object bindings and compiler output; the driver specifies the reproducible compiler commands. The Rocq driver preserves its compatibility-normalization boundary and runs both compilation and recursive checking. The residual driver records its own finite-check scope.

The machine, earlier derivations, numerical encoding, completeness arguments, and semantic-interface proposals belong to the ProveIt development and its credited predecessors. In particular, the existing shared-basis proposal is reused for its unrestricted adapter and unchanged Tarski tail; the semantic site interface comes from the earlier semantic-coverage work. The archive retains the relevant attribution and license notices. The new two-step S extension, current-source integration, and checked residual connection are identified separately. Independent review here means a second review within this research collaboration, not an external endorsement.

The deliverable preserves the exact machine and the existing external halting/refutation target. Its result is a smaller, easier-to-maintain implication development and a verified way to stop re-proving the same decoder prefix at every site. The final production adoption decision should rely on the exact component and endpoint gates above.

References

[1] ProveIt, *K215: 215 working states for ZFC consistency*, pinned commit `57541fe0cfe3af62b4cb58747d8d1d7c16b338d5`. https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/candidate-k215/README.md.

[2] ProveIt, *Twelfth K215 proof-simplification checkpoint*, same pinned commit. https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/phase12/README.md.

[3] ProveIt, *DominatedImplication.lean*, same pinned commit. https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/DominatedImplication.lean.

[4] ProveIt, *Lukasiewicz.lean*, same pinned commit. https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/Lukasiewicz.lean.

[5] ProveIt, *K215: one shared implication core and semantic site contracts*, archived shared-proof-core proposal; reviewed as prior work, not as maintained endpoint source. https://github.com/VladimirReshetnikov/ProveIt/tree/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/shared-proof-core.

[6] ProveIt, *K215 verification and simplification records*, same pinned commit. https://github.com/VladimirReshetnikov/ProveIt/tree/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215.

[7] ProveIt, *KDecoderPrefix215.lean*, same pinned commit. https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KDecoderPrefix215.lean.

[8] ProveIt, *Direct full seed*, prior proposal retained at the reviewed commit. https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/sharing-interfaces/calculus/DIRECT_FULL_SEED.md.